Modern University for Technology & Information

Faculty of Computers and Artificial Intelligence

Project Title:[Numerical Analysis Solver]

Course: (**BS-252)** Numerical Analysis

Supervised by: Dr. Rania Ahmed

Teaching Assistant's : [Abdalla Osama]

[Mina Yakoub]

Study Group: [G-1]

Team Members:

1. **[Ahmed Abdalla Ahmed]** - [109994]
2. **[Ahmed Sameh Mostafa]** - [111040]
3. **[Aboelela Magdy Sayed]** - [110732]

Contents

Chapter 1 : Introduction 3
1. Introduction 3
2. Objectives 3
3. Tools & Technologies Used 4
   1. Programming Language 4
   2. GUI Framework 4
   3. Libraries 4
   4. IDE 4
   5. Other tools 4
Chapter 2 : Application User Guide 5
Chapter 3 : Methods Implementation & Results 6
1. Chapter 1 : FINDING ROOTS OF POLYNOMIALS 6
   1. Bisection Method 6
   2. False position Method 8
   3. Newton'sMethod 10
   4. Simple Fixed point Method 12
   5. Secant Method 14
Chapter 2 : LINEAR ALGEBRAIC EQUATIONS 16
   1. Gauss Elimination 16
   2. LU Decomposition 18
   3. Cramer’s Rule 20
   4. Gauss-Jordan 22

Chapter 1 : Introduction

1. Introduction

Numerical analysis is a branch of mathematics concerned with developing algorithms to obtain approximate solutions for complex mathematical problems that cannot be solved analytically. This project presents the design and implementation of a Numerical Methods Solver capable of handling both non-linear equations and systems of linear equations. The application integrates multiple numerical techniques within a user-friendly interface, allowing users to compute accurate solutions efficiently while observing intermediate steps.

2. Objectives

- Design and implement an interactive application for solving numerical problems.
- Apply root-finding methods such as Bisection, False Position, Newton-Raphson, Secant, and Fixed Point.
- Solve systems of linear equations using methods like Gauss Elimination, LU Decomposition, Cramer's Rule, and Gauss-Jordan.
- Provide accurate and efficient computations with proper error handling.
- Display step-by-step iterations to enhance user understanding.

3. Tools & Technologies Used

1-Programming Language

JavaScript was used due to its flexibility and ability to build interactive applications, especially when combined with modern frameworks for graphical user interfaces. It allows efficient handling of user inputs, real-time computations, and dynamic result visualization. Additionally, JavaScript provides access to mathematical libraries that simplify the implementation of numerical methods.

2-GUI Framework

(HTML&CSS)

3-Libraries

Math.js was used for performing mathematical computations and handling expressions efficiently.

Html2pdf.js to covert the answers to pdf.

bootstrap

4-IDE

Visual Studio Code was used for writing, debugging, and managing the project.

5-Other tools

GitHub was used for version control and collaboration.

Chapter 2 : Application User Guide

Project Domain Link:
https://numerical-methods-solver.vercel.app/

How to Use the Application:

1-Select the required solving method from the Method dropdown list.

2-Enter the required input values.

3-Click the Solve button to perform calculations.

4-View the result and iteration steps.

5-Click Export to PDF to save the output.

1. Root-Finding Methods

For these methods, the user must enter a mathematical function in the f(x) field.

Examples:

x^2 - 4

sin(x) - x/2

log(x) - 1

Required inputs:

Bisection / False Position: lower bound xl and upper bound xu

Fixed Point / Newton: initial guess x_0

Secant: initial guesses x_0 and x_1

2. Linear System Methods

For matrix methods, enter the augmented matrix using the following format:

Separate rows with ;

Separate values with ,

Example:

2,1,5,8;1,-1,1,2;3,2,-1,3

Chapter 3 : Methods Implementation & Results

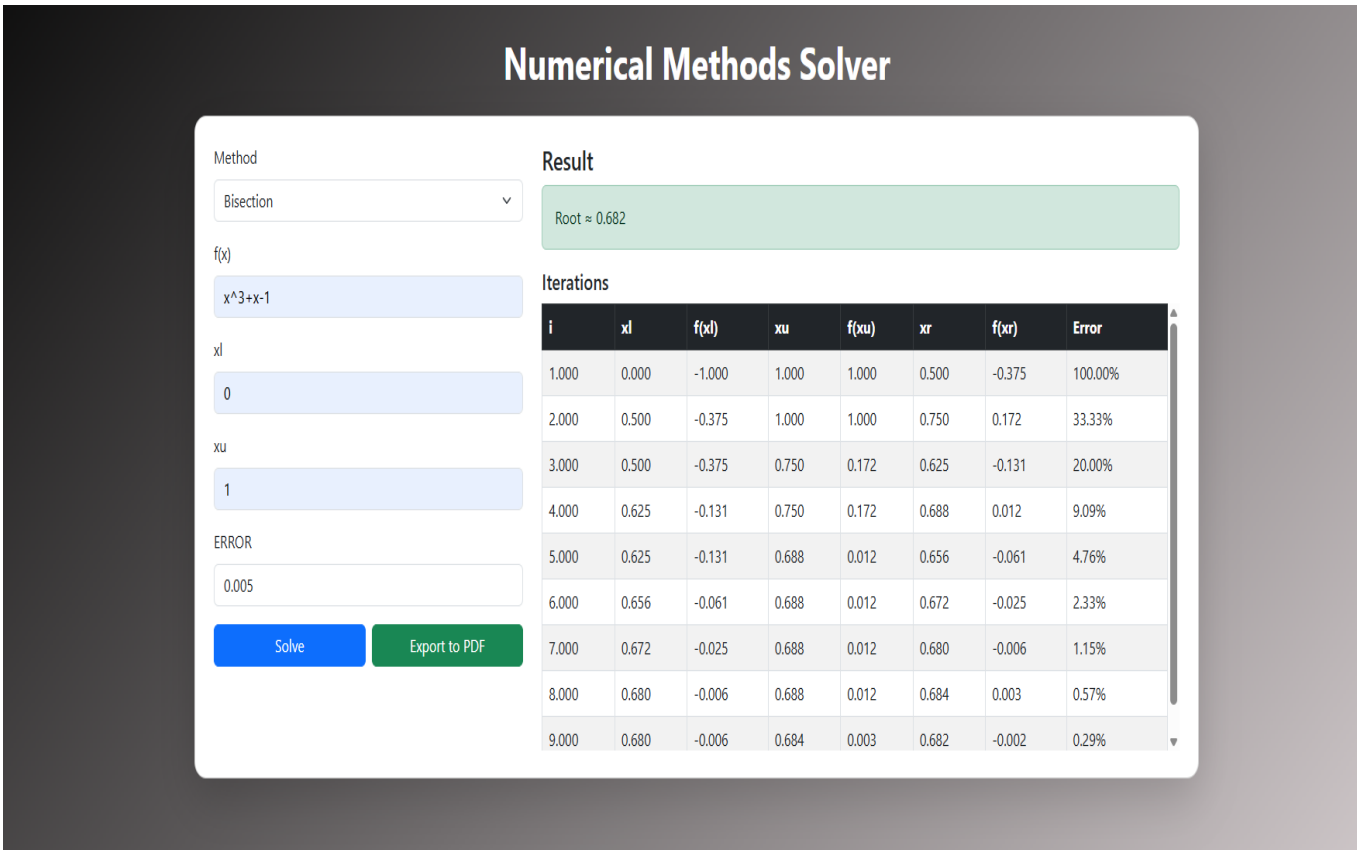
1. Chapter 1 : FINDING ROOTS OF POLYNOMIALS

1.Bisection Method

- **Page A: Code Snippet**

```
function solveBisection(expr, tol) {
    let xl = parseFloat(document.getElementById('xl').value)
    let xu = parseFloat(document.getElementById('xu').value)
    if (isNaN(xl) || isNaN(xu)) throw new Error('Enter valid xl and xu')
    let xr = 0, old = 0, err = 1, i = 1
    while (err > tol && i <= MAX_ITER) {
        xr = (xl + xu) / 2
        err = i > 1 ? Math.abs((xr - old) / (xr || 1)) : 1
        const fxl = evaluate(expr, xl)
        const fxu = evaluate(expr, xu)
        const fxr = evaluate(expr, xr)
        addRow([i, xl, fxl, xu, fxu, xr, fxr, (err * 100).toFixed(2) + '%'])
        if (fxl * fxr < 0) xu = xr; else xl = xr
        old = xr; i++
    }
    document.getElementById('result').textContent = `Root ≈ ${xr.toFixed(3)}`
}
```

Page B: Output (GUI Screenshot)



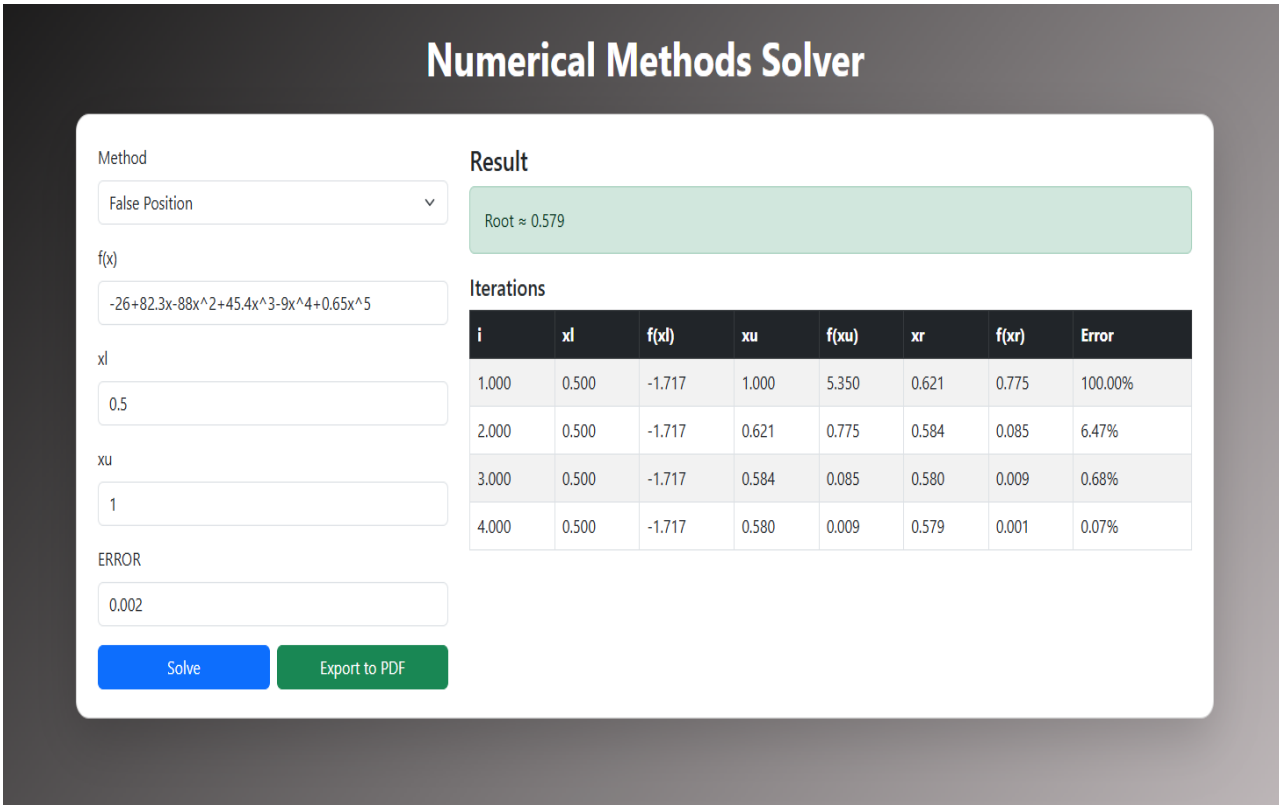
i	xl	f(xl)	xu	f(xu)	xr	f(xr)	Error
1.000	0.000	-1.000	1.000	1.000	0.500	-0.375	100.00%
2.000	0.500	-0.375	1.000	1.000	0.750	0.172	33.33%
3.000	0.500	-0.375	0.750	0.172	0.625	-0.131	20.00%
4.000	0.625	-0.131	0.750	0.172	0.688	0.012	9.09%
5.000	0.625	-0.131	0.688	0.012	0.656	-0.061	4.76%
6.000	0.656	-0.061	0.688	0.012	0.672	-0.025	2.33%
7.000	0.672	-0.025	0.688	0.012	0.680	-0.006	1.15%
8.000	0.680	-0.006	0.688	0.012	0.684	0.003	0.57%
9.000	0.680	-0.006	0.684	0.003	0.682	-0.002	0.29%

2.False position Method

- **Page A: Code Snippet**

```
function solveFalsePosition(expr, tol) {
    let xl = parseFloat(document.getElementById('xl').value)
    let xu = parseFloat(document.getElementById('xu').value)
    if (isNaN(xl) || isNaN(xu)) throw new Error('Enter valid xl and xu')
    let xr = 0, old = 0, err = 1, i = 1
    while (err > tol && i <= MAX_ITER) {
        const fxl = evaluate(expr, xl)
        const fxu = evaluate(expr, xu)
        xr = xu - (fxu * (xl - xu)) / (fxl - fxu)
        err = i > 1 ? Math.abs((xr - old) / (xr || 1)) : 1
        const fxr = evaluate(expr, xr)
        addRow([i, xl, fxl, xu, fxu, xr, fxr, (err * 100).toFixed(2) + '%'])
        if (fxl * fxr < 0) xu = xr; else xl = xr
        old = xr; i++
    }
    document.getElementById('result').textContent = `Root ≈ ${xr.toFixed(3)}`
}
```

- **Page B: Output (GUI Screenshot)**



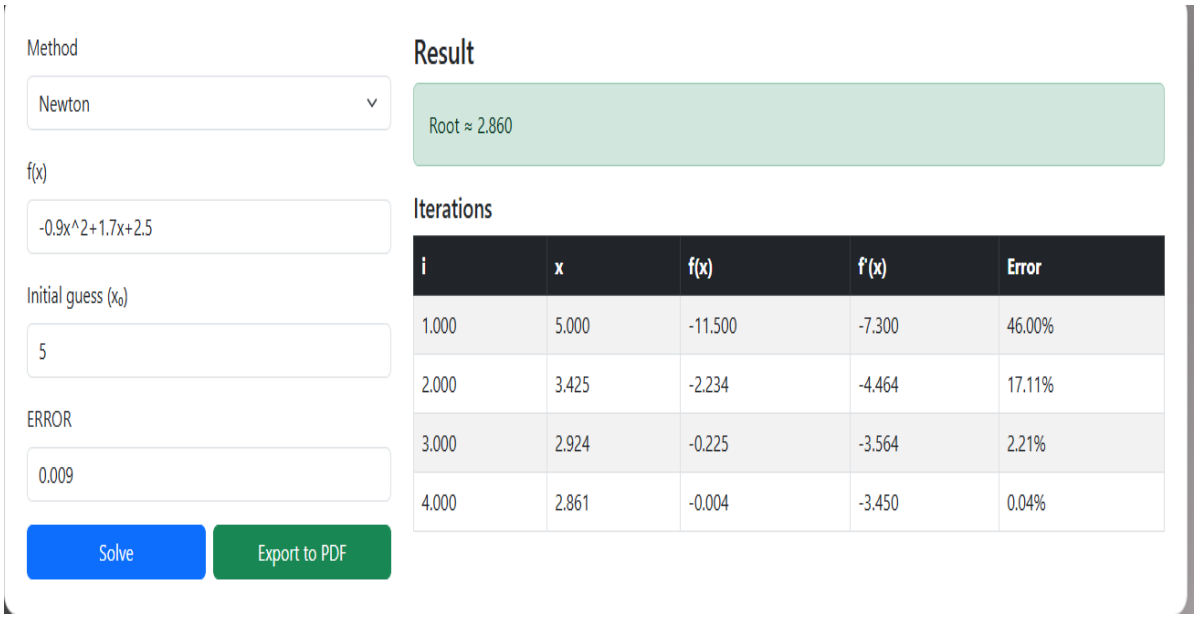
i	xl	f(xl)	xu	f(xu)	xr	f(xr)	Error
1.000	0.500	-1.717	1.000	5.350	0.621	0.775	100.00%
2.000	0.500	-1.717	0.621	0.775	0.584	0.085	6.47%
3.000	0.500	-1.717	0.584	0.085	0.580	0.009	0.68%
4.000	0.500	-1.717	0.580	0.009	0.579	0.001	0.07%

3. Newton'sMethod

Page A: Code Snippet

```
function solveNewton(expr, tol) {
    let x = parseFloat(document.getElementById('x0').value)
    if (isNaN(x)) throw new Error('Enter a valid initial guess')
    const h = 1e-5
    let err = 1, i = 1
    while (err > tol && i <= MAX_ITER) {
        const fx = evaluate(expr, x)
        const dfx = (evaluate(expr, x + h) - evaluate(expr, x - h)) / (2 * h)
        if (dfx === 0) throw new Error('Derivative became zero')
        const xn = x - fx / dfx
        err = Math.abs((xn - x) / (xn || 1))
        addRow([i, x, fx, dfx, (err * 100).toFixed(2) + '%'])
        x = xn; i++
    }
    document.getElementById('result').textContent = `Root ≈ ${x.toFixed(3)}`
}
```

Page B: Output (GUI Screenshot)



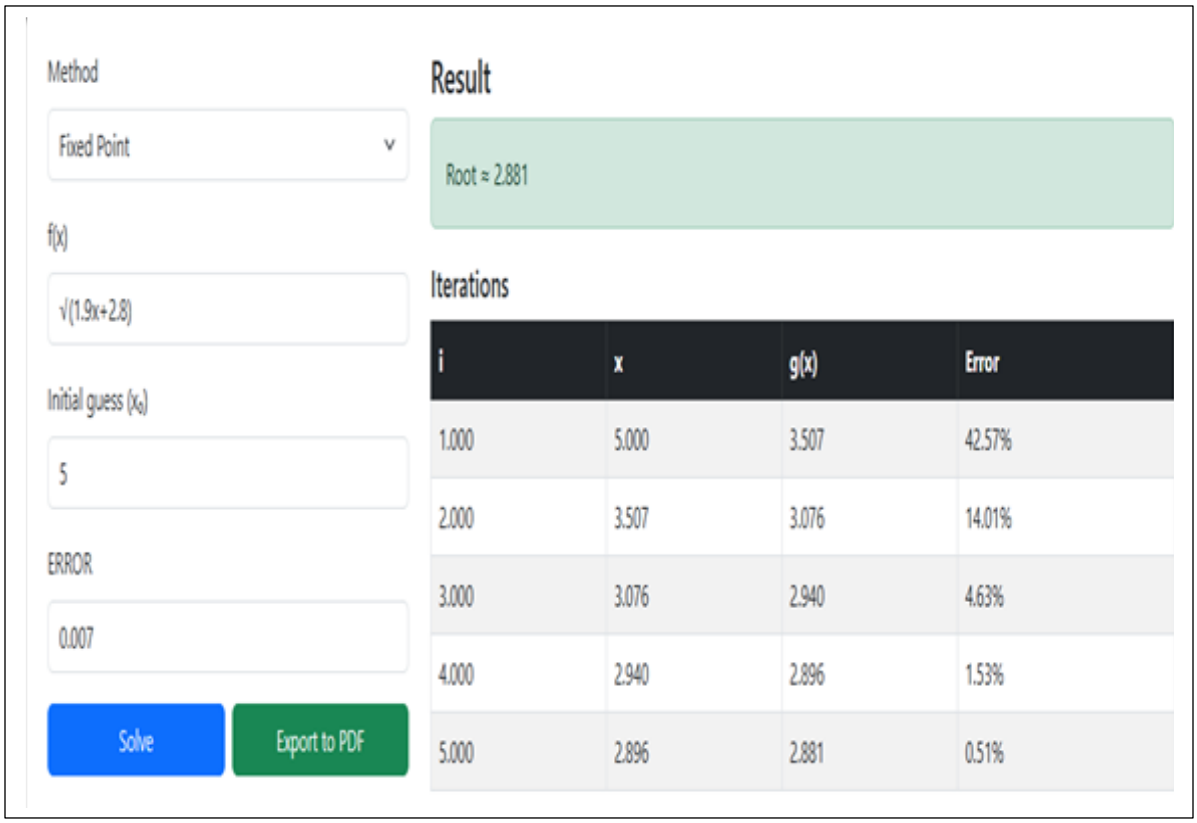
i	x	f(x)	f'(x)	Error
1.000	5.000	-11.500	-7.300	46.00%
2.000	3.425	-2.234	-4.464	17.11%
3.000	2.924	-0.225	-3.564	2.21%
4.000	2.861	-0.004	-3.450	0.04%

4.Simple Fixed point Method

Page A: Code Snippet

```
function solveFixedPoint(expr, tol) {
    let x = parseFloat(document.getElementById('x0').value)
    if (isNaN(x)) throw new Error('Enter a valid initial guess')
    let err = 1, i = 1
    while (err > tol && i <= MAX_ITER) {
        const xn = evaluate(expr, x)
        err = Math.abs((xn - x) / (xn || 1))
        addRow([i, x, xn, (err * 100).toFixed(2) + '%'])
        x = xn; i++
    }
    document.getElementById('result').textContent = `Root ≈ ${x.toFixed(3)}`
}
```

Page B: Output (GUI Screenshot)



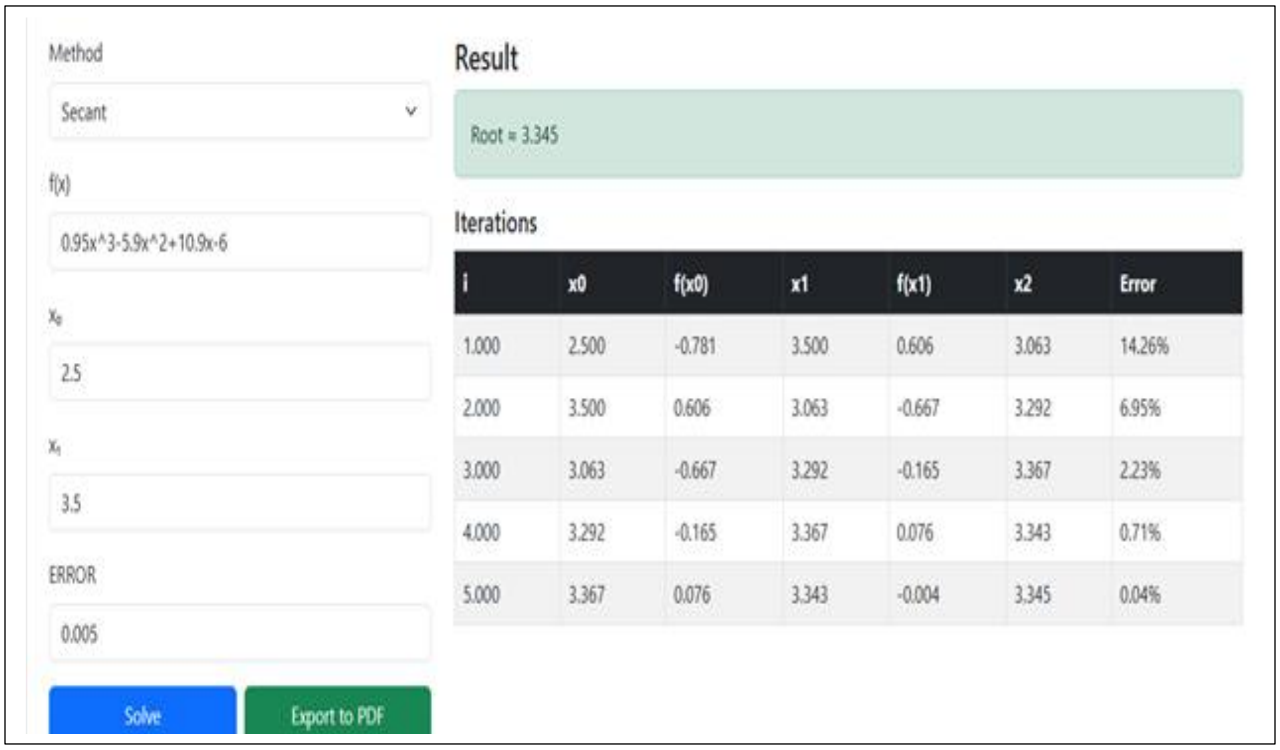
i	x	g(x)	Error
1.000	5.000	3.507	42.57%
2.000	3.507	3.076	14.01%
3.000	3.076	2.940	4.63%
4.000	2.940	2.896	1.53%
5.000	2.896	2.881	0.51%

5.Secant Method

Page A: Code Snippet

```
function solveSecant(expr, tol) {
    let x0 = parseFloat(document.getElementById('x0').value)
    let x1 = parseFloat(document.getElementById('x1').value)
    if (isNaN(x0) || isNaN(x1)) throw new Error('Enter valid x0 and x1')
    let err = 1, i = 1
    while (err > tol && i <= MAX_ITER) {
        const f0 = evaluate(expr, x0)
        const f1 = evaluate(expr, x1)
        if (f0 === f1) throw new Error('Division by zero (f(x0) == f(x1))')
        const x2 = x1 - f1 * (x0 - x1) / (f0 - f1)
        err = Math.abs((x2 - x1) / (x2 || 1))
        addRow([i, x0, f0, x1, f1, x2, (err * 100).toFixed(2) + '%'])
        x0 = x1; x1 = x2; i++
    }
    document.getElementById('result').textContent = `Root ≈ ${x1.toFixed(3)}`
}
```

Page B: Output (GUI Screenshot)



i	x0	f(x0)	x1	f(x1)	x2	Error
1.000	2.500	-0.781	3.500	0.606	3.063	14.26%
2.000	3.500	0.606	3.063	-0.667	3.292	6.95%
3.000	3.063	-0.667	3.292	-0.165	3.367	2.23%
4.000	3.292	-0.165	3.367	0.076	3.343	0.71%
5.000	3.367	0.076	3.343	-0.004	3.345	0.04%

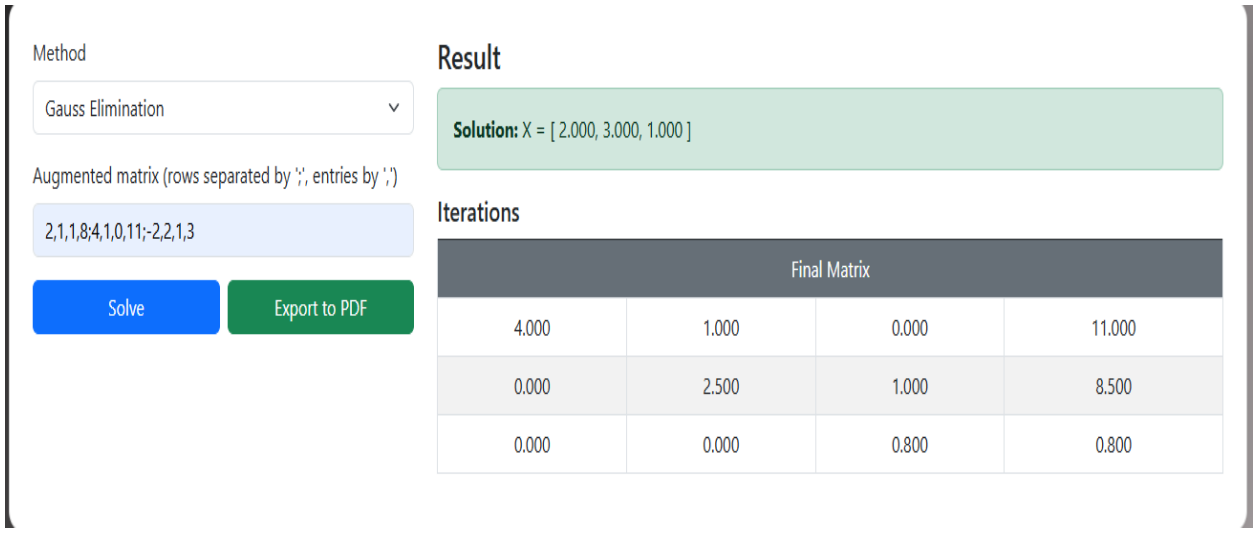
Chapter 2 : LINEAR ALGEBRAIC EQUATIONS

1.Gauss Elimination

Page A: Code Snippet

```
export function solveGauss() {
    const input = document.getElementById('matrix').value;
    if (!input) throw new Error('Enter matrix');
    let A = input.split(';').map(row => row.split(',').map(Number));
    let n = A.length;
    for (let i = 0; i < n; i++) {
        for (let k = i + 1; k < n; k++) {
            let factor = A[k][i] / A[i][i];
            for (let j = i; j <= n; j++) {
                A[k][j] -= factor * A[i][j];
            }
        }
    }
    let X = Array(n).fill(0);
    for (let i = n - 1; i >= 0; i--) {
        let sum = 0;
        for (let j = i + 1; j < n; j++) {
            sum += A[i][j] * X[j];
        }
        X[i] = (A[i][n] - sum) / A[i][i];
    }

    displayGaussMatrix(A, X);
}
```

Page B: Output (GUI Screenshot)



2.LUDecomposition

```
function calculateDet(matrix) {
    let n = matrix.length;
    let A = matrix.map(row => [...row]);
    let det = 1;
    let swaps = 0;

    for (let i = 0; i < n; i++) {
        let maxRow = i;
        for (let k = i + 1; k < n; k++) {
            if (Math.abs(A[k][i]) > Math.abs(A[maxRow][i])) maxRow = k;
        }
        if (maxRow !== i) {
            [A[i], A[maxRow]] = [A[maxRow], A[i]];
            swaps++;
        }
        if (Math.abs(A[i][i]) < 1e-10) return 0;
        for (let k = i + 1; k < n; k++) {
            let factor = A[k][i] / A[i][i];
            for (let j = i; j < n; j++) { A[k][j] -= factor * A[i][j]; }
        }
        det *= A[i][i];
    }
    return swaps % 2 === 0 ? det : -det;
}

export function solveCramer() {
    const matrixInput = document.getElementById('matrix').value;
    if (!matrixInput) throw new Error('Please enter the matrix');

    const matrix = matrixInput.split(';').map(row => row.split(',').map(Number));
    const n = matrix.length;

    const A = []; const B = [];
    for (let i = 0; i < n; i++) {
        A[i] = matrix[i].slice(0, n);
        B[i] = matrix[i][n];
    }
```

```
    const Y = Array(n).fill(0);
    for (let i = 0; i < n; i++) {
        let sum = 0;
        for (let j = 0; j < i; j++) sum += L[i][j] * Y[j];
        Y[i] = B[i] - sum;
    }

    const X = Array(n).fill(0);
    for (let i = n - 1; i >= 0; i--) {
        let sum = 0;
        for (let j = i + 1; j < n; j++) sum += U[i][j] * X[j];
        X[i] = (Y[i] - sum) / U[i][i];
    }

    displayLUMatrix(L, U, X);
}
```

Method

LU Decomposition

Augmented matrix (rows separated by ';', entries by ',')

2,1,1,8;4,1,0,11;-2,2,1,3

Solve | Export to PDF

Result

Solution: X = [2.000, 3.000, 1.000]

Iterations

Matrix L		
1.000	0.000	0.000
2.000	1.000	0.000
-1.000	-3.000	1.000
Matrix U		
2.000	1.000	1.000
0.000	-1.000	-2.000
0.000	0.000	-4.000

3.Cramer's Rule

Page A: Code Snippet

```javascript
function calculateDet(matrix) {
    let n = matrix.length;
    let A = matrix.map(row => [...row]);
    let det = 1;
    let swaps = 0;

    for (let i = 0; i < n; i++) {
        let maxRow = i;
        for (let k = i + 1; k < n; k++) {
            if (Math.abs(A[k][i]) > Math.abs(A[maxRow][i])) maxRow = k;
        }
        if (maxRow !== i) {
            [A[i], A[maxRow]] = [A[maxRow], A[i]];
            swaps++;
        }
        if (Math.abs(A[i][i]) < 1e-10) return 0;
        for (let k = i + 1; k < n; k++) {
            let factor = A[k][i] / A[i][i];
            for (let j = i; j < n; j++) { A[k][j] -= factor * A[i][j]; }
        }
        det *= A[i][i];
    }
    return swaps % 2 === 0 ? det : -det;
}

export function solveCramer() {
    const matrixInput = document.getElementById('matrix').value;
    if (!matrixInput) throw new Error('Please enter the matrix');

    const matrix = matrixInput.split(';').map(row => row.split(',').map(Number));
    const n = matrix.length;

    const A = []; const B = [];
    for (let i = 0; i < n; i++) {
        A[i] = matrix[i].slice(0, n);
        B[i] = matrix[i][n];
    }
```

```javascript
    for (let i = 0; i < n; i++) {
        let temp = A.map(row => [...row]);
        for (let j = 0; j < n; j++) {
            temp[j][i] = B[j];
        }
        const d_i = calculateDet(temp);
        determinants.push(d_i);
        X.push(d_i / detA);
    }

    displayCramer(detA, determinants, X);
}
```

Page B: Output (GUI Screenshot)

Method

Cramer's Rule

Augmented matrix (rows separated by ';', entries by ',')

2,1,1,8;4,1,0,11;-2,2,1,3

Solve | Export to PDF

Result

Solution: X = [2.000, 3.000, 1.000]

Iterations

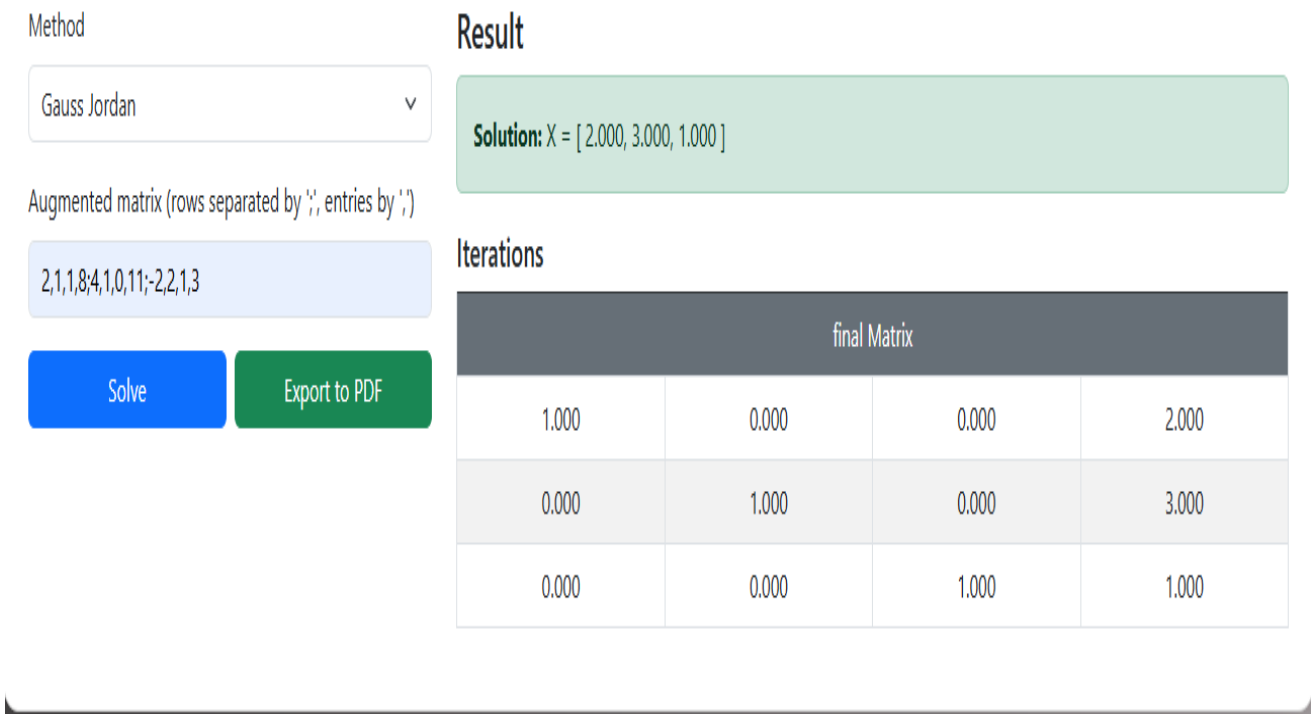
Cramer's Rule Results				
Main Determinant (Det A) = **8.000**				
Det A_1	16.000	/	8.000	**2.000**
Det A_2	24.000	/	8.000	**3.000**
Det A_3	8.000	/	8.000	**1.000**

4.Gauss-Jordan

Page A: Code Snippet

```
export function solveGaussJordan() {
    const matrixInput = document.getElementById('matrix').value;
    if (!matrixInput) throw new Error('Please enter the matrix');

    const matrix = matrixInput.split(';').map(row => row.split(',').map(Number));
    const n = matrix.length;
    let A = matrix.map(row => [...row]);

    for (let i = 0; i < n; i++) {

        let maxRow = i;
        for (let k = i + 1; k < n; k++) {
            if (Math.abs(A[k][i]) > Math.abs(A[maxRow][i])) maxRow = k;
        }
        [A[i], A[maxRow]] = [A[maxRow], A[i]];

        const pivot = A[i][i];
        if (Math.abs(pivot) < 1e-10) throw new Error('Matrix is singular or nearly singular');

        for (let j = i; j <= n; j++) {
            A[i][j] /= pivot;
        }

        for (let k = 0; k < n; k++) {
            if (k !== i) {
                const factor = A[k][i];
                for (let j = i; j <= n; j++) {
                    A[k][j] -= factor * A[i][j];
                }
            }
        }
    }

    const X = A.map(row => row[n]);
    displayJordanMatrix(A, X);
}
```

- **Page B: Output (GUI Screenshot)**



Summary & Conclusion

The web application that I made, my numerical methods solver, was built using HTML, CSS, and JavaScript. I was able to combine all types of numerical methods (bisection method, false position method, fixed point method, Newton method, secant method, gauss elimination method, LU decomposition method, gauss-Jordan method, Cramer's rule) into one web application.

I developed the user interface so that users can easily type in their equation of a function or matrix and see step-by-step iteration results of solving that equation. Additionally, the web application can produce PDFs of the results

Using the html2pdf.script

I picked up some practical knowledge about how numerical methods actually work, as well as developed a stronger ability to understand how iterative algorithms find their solution. I developed greater skill relating to JavaScript programming, DOM manipulation, input validation, and organizing an entire web application project.